

Report:Digital Clock

RUGVED KSHIRSAGAR

Contents

1	Digital Clock Prototype	2
1.1	Introduction	2
1.2	Objectives	2
1.3	System Architecture	2
1.4	Hardware Components	3
1.5	Working Principle	3
1.6	Software Flow	4
1.7	Implementation and Debugging	4
1.8	Firmware Development	4
1.9	Prototype	5
1.10	Fusion 360 Enclosure Design	5
1.11	Results	7
1.12	Conclusion	7

Chapter 1

Digital Clock Prototype

1.1 Introduction

The primary objective of this project was to design and implement a standalone digital clock using the ATmega328P microcontroller. The project integrated concepts of embedded systems, digital electronics and hardware interfacing into a functional real-time application. The clock displays time in the HH:MM format using a four-digit seven-segment display and allows the user to set the current time through dedicated push buttons.

The project also involved hardware debugging, software development, display multiplexing and enclosure design using Autodesk Fusion 360, thereby providing complete exposure to the embedded system development cycle.

1.2 Objectives

The objectives of the project were:

- Design a standalone embedded system using the ATmega328P.
- Interface a four-digit seven-segment display using GPIO pins.
- Implement software-based timekeeping.
- Allow manual time setting through push buttons.
- Develop a compact enclosure using Autodesk Fusion 360.

1.3 System Architecture

The digital clock consists of a standalone ATmega328P microcontroller, a four-digit seven-segment display, three push buttons, an external 16 MHz crystal oscillator and the supporting passive components. The microcontroller executes the timekeeping algorithm, scans the display using multiplexing and monitors user inputs.

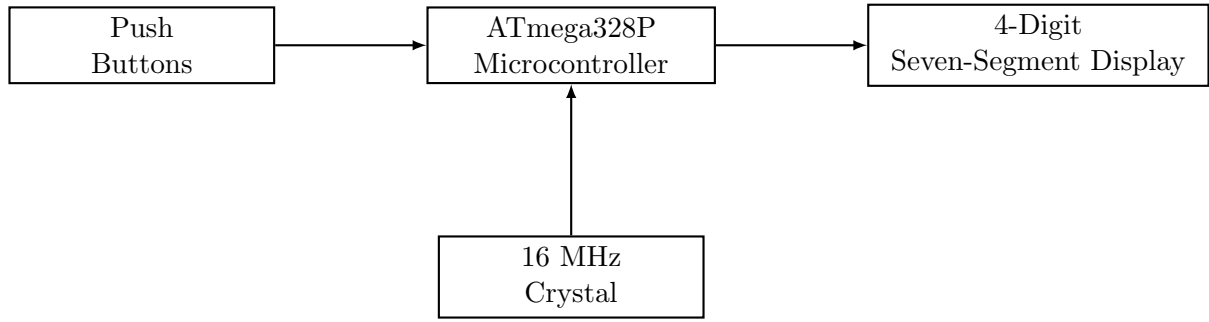


Figure 1.1: Block Diagram of the Digital Clock

1.4 Hardware Components

Table 1.1: Hardware Components Used

Component	Quantity
ATmega328P Microcontroller	1
Four-Digit Seven Segment Display	1
16 MHz Crystal Oscillator	1
Push Buttons	3
330 Ω Resistors	8
100 nF Capacitors	2
220 pF Capacitors	2
5 V Power Supply	1
Breadboard/PCB	1

1.5 Working Principle

The ATmega328P maintains the current time through software counters driven by the external 16 MHz crystal oscillator. The four-digit seven-segment display is controlled using GPIO pins with multiplexing, where only one digit is activated at a time. Since the switching frequency is sufficiently high, persistence of vision makes all digits appear continuously illuminated.

Three push buttons provide user interaction.

- Hour Button – increments the hour.
- Minute Button – increments the minute.
- Start Button – starts normal clock operation after setting the time.

1.6 Software Flow

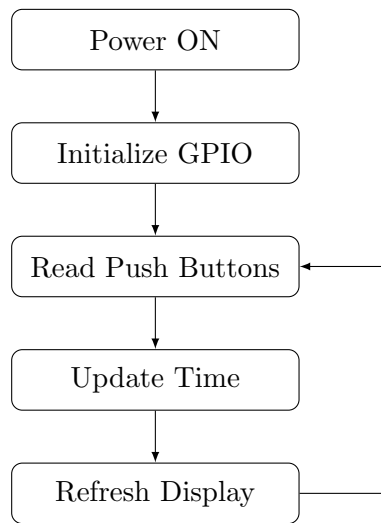


Figure 1.2: Software Flow of the Digital Clock

1.7 Implementation and Debugging

The prototype was initially developed and tested using the Arduino development environment before programming the standalone ATmega328P microcontroller. GPIO pins were configured to control the display segments and read the push-button inputs.

Several practical challenges were encountered during implementation, including external crystal oscillator configuration, hardware wiring, stable display multiplexing and program uploading to the standalone microcontroller. These issues were resolved through systematic hardware verification and software debugging until reliable clock operation was achieved.

1.8 Firmware Development

The firmware for the digital clock was developed in Embedded C using the Arduino IDE and programmed into the standalone ATmega328P microcontroller. The software initializes the GPIO pins, continuously refreshes the four-digit seven-segment display using multiplexing, reads the Hour, Minute and Start push buttons, and updates the displayed time every second.

Listing 1.1 presents a representative portion of the firmware illustrating the initialization and main execution loop.

Listing 1.1: Main Firmware Structure

```
1  const byte segPins[7]    = {2,3,4,5,6,7,8};
2  const byte digitPins[4] = {9,10,11,12};
3
4  const byte hourBtn      = A0;
5  const byte minuteBtn    = A1;
6  const byte startBtn     = A2;
7
8  void setup()
9  {
10     for(int i=0;i<7;i++)
11         pinMode(segPins[i],OUTPUT);
12
13     for(int i=0;i<4;i++)
14         pinMode(digitPins[i],OUTPUT);
15 }
```

```

16     pinMode(hourBtn, INPUT_PULLUP);
17     pinMode(minuteBtn, INPUT_PULLUP);
18     pinMode(startBtn, INPUT_PULLUP);
19 }
20
21 void loop()
22 {
23     readButtons();
24     updateClock();
25     refreshDisplay();
26 }

```

The complete firmware consists of multiple functions responsible for GPIO initialization, button handling, software-based timekeeping and display multiplexing. Only the main program structure is included here for brevity.

1.9 Prototype

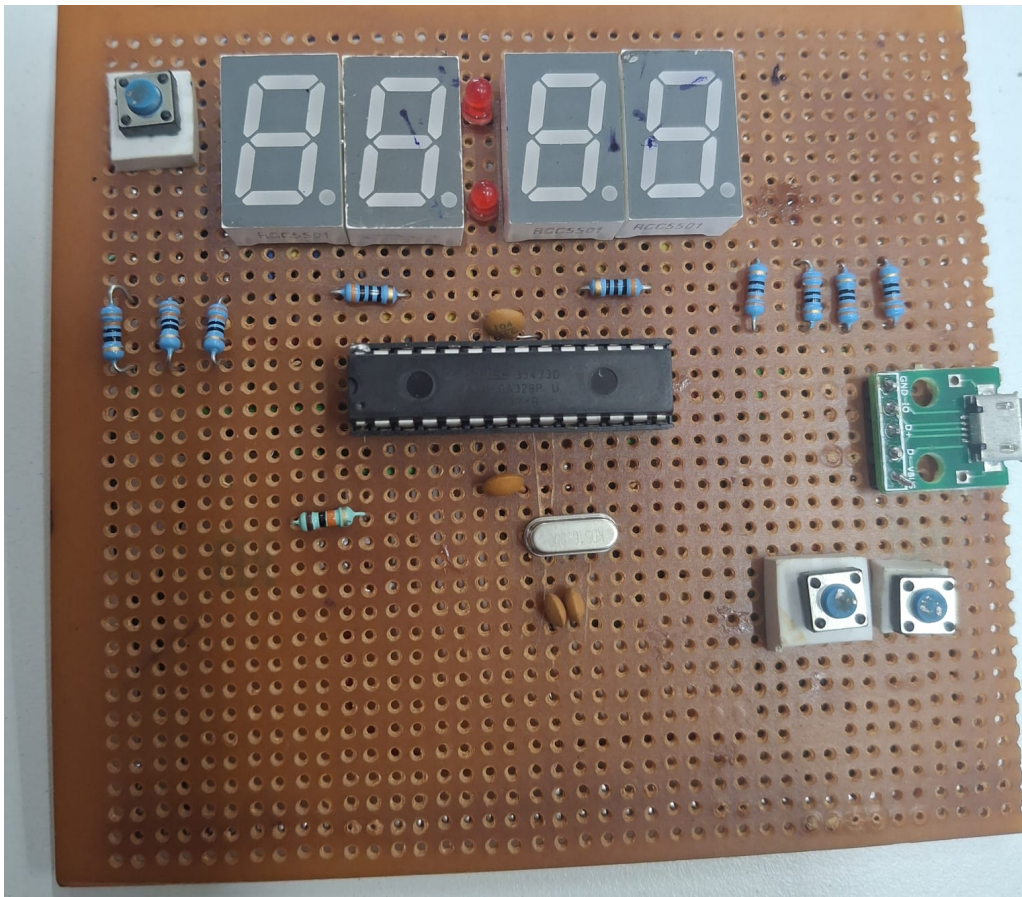


Figure 1.3: Completed Digital Clock Prototype

1.10 Fusion 360 Enclosure Design

To improve the appearance and protection of the prototype, a custom enclosure was designed using Autodesk Fusion 360. The enclosure was modeled to accommodate the display, push buttons and power connections while providing adequate structural support for the electronic components. The CAD model can be directly used for fabrication using additive manufacturing techniques such as 3D printing.

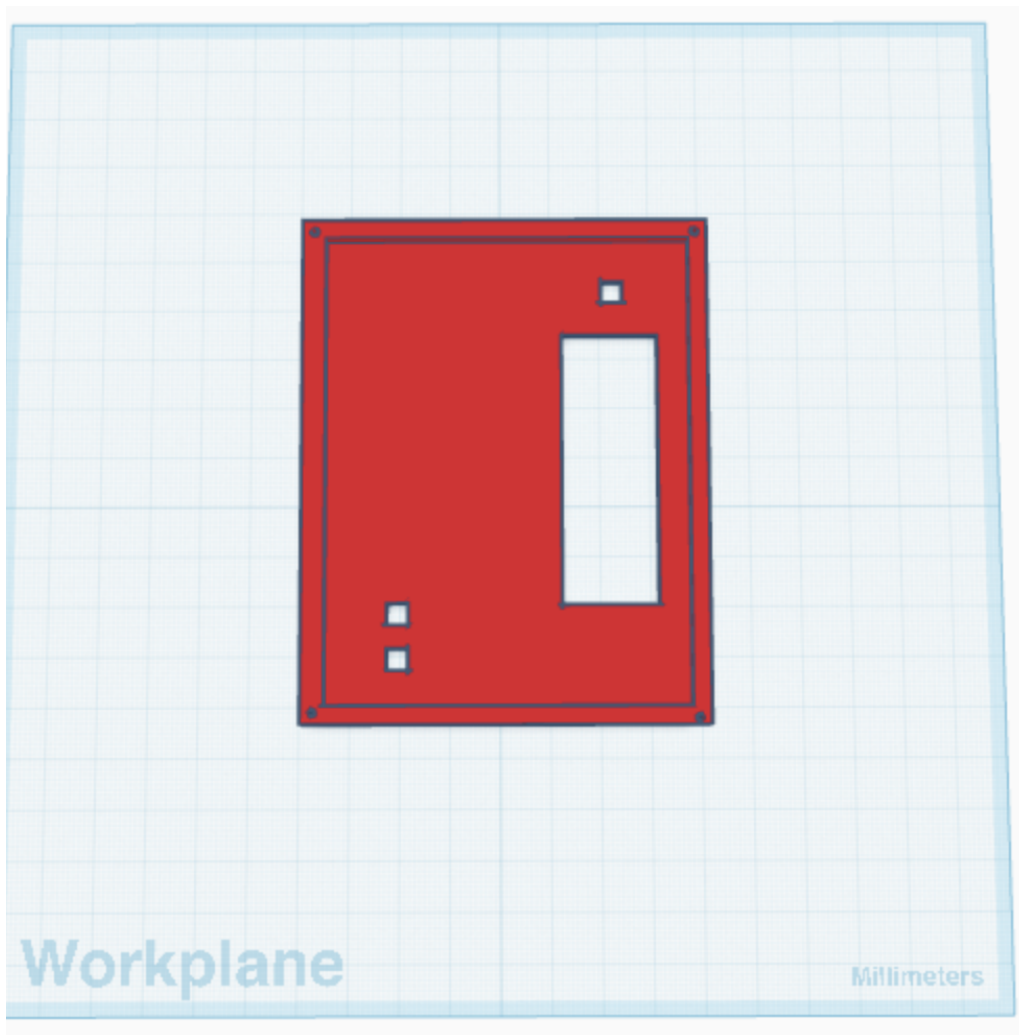


Figure 1.4: Fusion 360 Model of the Enclosure-Top

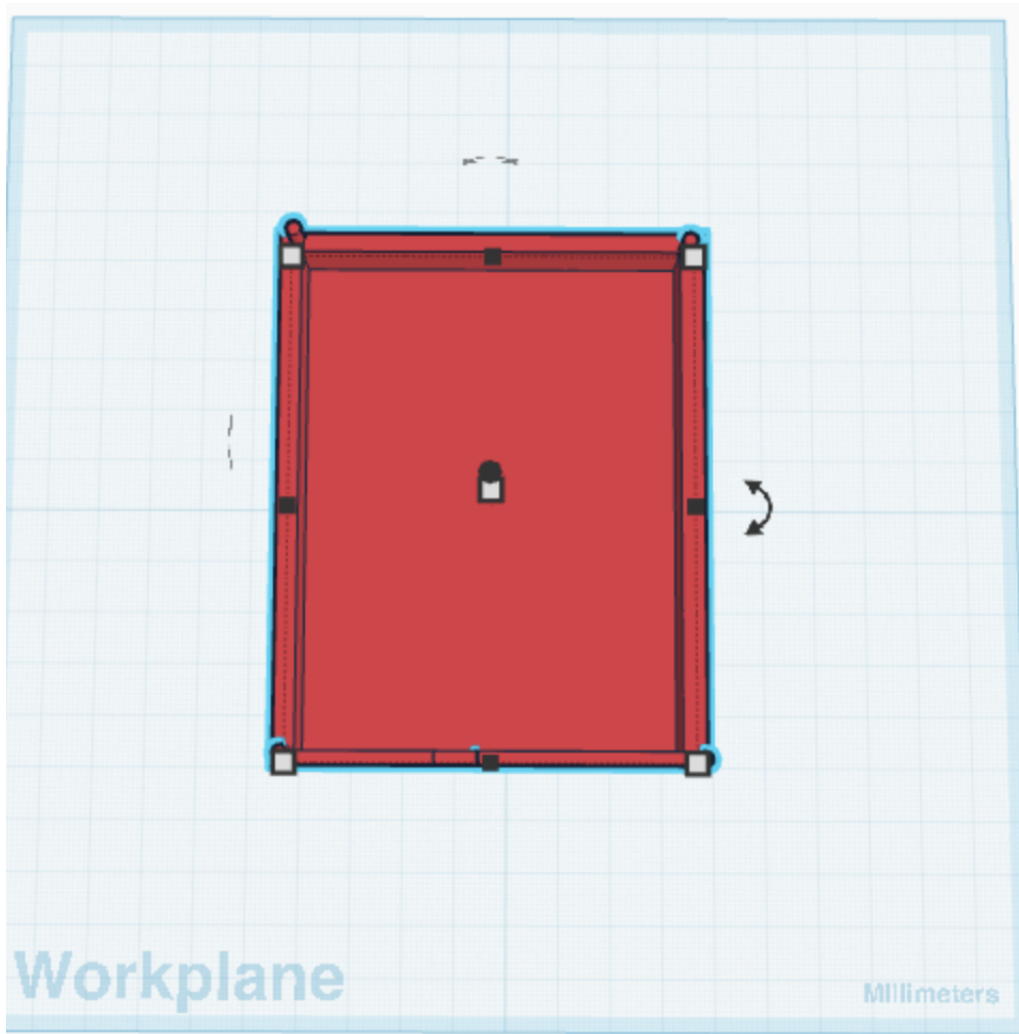


Figure 1.5: Bottom

1.11 Results

The digital clock successfully displayed time in the HH:MM format using a standalone ATmega328P microcontroller. The Hour, Minute and Start buttons functioned as intended, allowing the user to configure and operate the clock. The multiplexed display produced stable, flicker-free output, while the external crystal oscillator ensured reliable timekeeping. The enclosure designed using Fusion 360 complemented the hardware by providing a compact and organized mechanical structure.

1.12 Conclusion

The digital clock project successfully integrated embedded programming, digital electronics and mechanical design into a complete embedded system. The project strengthened practical understanding of GPIO interfacing, display multiplexing, hardware debugging and standalone microcontroller implementation. In addition to achieving the intended functionality, the project demonstrated the complete workflow involved in designing, implementing and packaging an embedded hardware prototype.